SOFTWARE REQUIREMENTS SPECIFICATION

For

Hostel Management System

Prepared by

Anvay Sonare

Roll No. 31 | Section B4 | Batch B2

Software Engineering and Project Management (SEPM)

Table of Contents

1. Introduction 4
1.1 Purpose 4
1.2 Document Conventions 4
1.3 Intended Audience and Reading Suggestions 4
1.4 Project Scope 4
1.5 References 5

2. Overall Description 6
2.1 Product Perspective 6
2.2 Product Features 6
2.3 User Classes and Characteristics 6
2.4 Operating Environment 6
2.5 Design and Implementation Constraints 7
2.6 Assumptions and Dependencies 7

3. System Features (Functional Requirements) 8
3.1 Registration, Login and Profile — Actors: Student, Admin 8
3.2 Room Application and Allocation — Actors: Student, Admin 8
3.3 Fee Payment — Actor: Student 8
3.4 Mess Management — Actors: Student, Admin 9
3.5 Administration and Reporting — Actor: Admin 9

4. External Interface Requirements 10
4.1 User Interfaces 10
4.2 Hardware Interfaces 10
4.3 Software Interfaces 10
4.4 Communication Interfaces 10

5. Non-Functional Requirements 11
5.1 Performance 11
5.2 Security 11
5.3 Reliability and Availability 11
5.4 Usability 11
5.5 Maintainability and Scalability 11

6. System Design — UML Models 12
6.1 Use Case Diagram 12
6.2 Activity Diagrams 13
6.2.1 Activity Diagram — Room Application and Allocation 14
6.2.2 Activity Diagram — Fee Payment 15
6.3 Class Diagram 16
6.4 Sequence Diagram 17
6.5 State Diagram 18
6.6 Deployment Diagram 21

6.7 Component Diagram....22

7. Cost Estimation....24

7.1 Estimation Technique....24

7.2 Size Estimate....24

7.3 Effort, Schedule and Team Size....24

7.4 Estimated Cost....25

7.5 Relationship to the Project Schedule....25

8. Project Schedule....26

8.1 Key Milestones....26

Appendix A: Glossary of Terms....27

1. Introduction

1.1 Purpose

This document specifies the software requirements for the Hostel Management System (HMS), a web- and mobile-based application that automates room allocation, fee collection and mess operations for a college hostel. It is written for the project guide, the development team, and anyone evaluating the design before implementation begins. The requirements, interface definitions and system models described here form the basis for the design and testing work that follows.

1.2 Document Conventions

Each functional requirement is numbered REQ-<module>.<sequence> and uses the word “shall” to mark a mandatory system capability. Section numbers follow a decimal scheme (1, 1.1, 1.1.1). Diagrams are numbered by the section in which they appear — Figure 6.1, 6.2, and so on — and are referred to by that number in the surrounding text. Class, controller and table names taken directly from the design are written as they appear in the diagrams, e.g. RoomController, Fee, Hostel_DB.

1.3 Intended Audience and Reading Suggestions

This SRS is meant for the project guide/evaluator, the student developer(s) responsible for building the Room, Fee, Mess and Admin modules, and anyone who maintains the system afterwards. A reader interested only in what the system does should read Sections 2 and 3; a reader interested in how it is built should go directly to Section 6, where the UML models are presented; Section 7 and 8 cover cost and schedule.

1.4 Project Scope

The Hostel Management System replaces the manual, register-based process a warden currently uses to allot rooms, track mess bills and collect fees. In scope: student registration and login, room application and allocation, fee invoicing and payment, mess menu updates and monthly bill generation, and administrative functions for managing users, assigning rooms and generating reports.

Out of scope: routing payments through a third-party banking gateway (the system processes and records payment within its own Fee module rather than calling an external API), hostel admission/counselling workflows, and transport or library modules — none of these appear in the modelled use cases and they are not addressed by this version of the system.

1.5 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- Object Management Group, OMG Unified Modeling Language (UML), Version 2.5.1.
- Use case, activity, class, sequence, state, deployment and component diagrams prepared for this project (SEPM Practical No. 2–5), referenced throughout Section 6.
- Project Gantt chart prepared for this project (SEPM Practical No. 7), referenced in Section 8.

2. Overall Description

2.1 Product Perspective

HMS is a new, self-contained system; it does not replace or interface with any existing hostel software. It follows a layered client–server architecture, shown in the component diagram (Figure 6.7): a Client Layer (a web browser for the admin and a mobile app for students), an Application Logic Layer made up of an Authentication Module and the Room, Fee and Mess controllers, and a Data Access Layer that talks to a single MySQL database, Hostel_DB, through a DB Connection Manager. The deployment diagram (Figure 6.6) places the application layer on an Apache Tomcat server and the database on a separate database server, connected over JDBC.

2.2 Product Features

At a high level, HMS supports:

- Student self-registration, login and profile management
- Room availability checking, application, and admin-side approval or rejection
- Fee invoicing, payment and payment-receipt verification
- Mess menu updates, daily consumption tracking and monthly bill generation
- Admin-side user management, room assignment and report generation

2.3 User Classes and Characteristics

Two user classes interact with the system, matching the two actors in the use case diagram (Figure 6.1):

Student — a resident or prospective resident of the hostel. Assumed to have basic familiarity with mobile apps and web forms; uses the system to register, apply for a room, pay fees and check the mess bill.

Admin (hostel warden / office staff) — manages day-to-day hostel operations: approving or rejecting applications, assigning rooms, updating the mess menu and generating reports. Assumed to be comfortable with a browser-based dashboard but not necessarily technical.

2.4 Operating Environment

- Client side: Android/iOS for the student-facing mobile app; Windows/macOS with a standard web browser for the admin workstation (Figure 6.6).
- Server side: an application server running Ubuntu Linux, with Apache Tomcat as the execution environment for the Auth, Room and Fee controllers.

- Database side: a separate server running CentOS Linux with MySQL 8.0, hosting Hostel_DB.
- Network: client traffic reaches the application server over HTTPS (port 443) through a firewall/load balancer; the application server talks to the database over JDBC.

2.5 Design and Implementation Constraints

- The backend is organised as independent controller components (Auth, Room, Fee, Mess), each exposed through its own interface — Auth API, Room API, Fee API, Mess API — so a client never queries the database directly (Figure 6.7).
- MySQL 8.0 is the only supported database engine; every controller reaches it through a single DB Connection Manager.
- Only HTTPS is permitted between client and server; the firewall/load balancer in Figure 6.6 is a required part of the deployment, not an optional add-on.
- The system must support the Student and Admin roles with different permissions against the same underlying controllers.

2.6 Assumptions and Dependencies

- Room inventory (room number, capacity, type) and at least one warden account exist in the database before go-live; HMS does not create hostel/room records out of nothing.
- Students have a working email address, since fee-payment confirmation is sent by email.
- One deployment of HMS serves one hostel/warden; the Hostel class permits more than one hostel record, but multi-hostel administration is not separately covered by the modelled use cases.
- A functioning SMTP service is reachable from the application server for outgoing confirmation emails.

3. System Features (Functional Requirements)

Requirements are grouped by module. Each module corresponds to a controller in the design (Figures 6.3, 6.6 and 6.7) and to a cluster of use cases in Figure 6.1.

3.1 Registration, Login and Profile — Actors: Student, Admin

REQ-1.1: The system shall allow a new student to register by providing name, email, phone number, course and password.

REQ-1.2: The system shall allow a registered student or an admin to log in with a valid email/username and password, and shall reject invalid credentials.

REQ-1.3: The system shall allow a logged-in student to update their profile information.

REQ-1.4: The system shall allow a logged-in user to log out and end the session.

3.2 Room Application and Allocation — Actors: Student, Admin

REQ-2.1: The system shall allow a student to check current room availability before applying.

REQ-2.2: The system shall allow a student to submit a room application and, optionally, select a room preference as part of that application.

REQ-2.3: The system shall allow the admin to review a submitted application and either approve or reject it.

REQ-2.4: The system shall notify the student that no room is available whenever an application is rejected.

REQ-2.5: The system shall concurrently generate a fee invoice for the student and update the room's status in the database once an application is approved.

REQ-2.6: The system shall notify the warden whenever a room's status changes.

REQ-2.7: The system shall confirm final room allocation to the student only after the generated fee invoice has been paid and the payment verified.

REQ-2.8: The system shall allow the admin to assign a room to a student directly; this action shall include a room-availability check.

3.3 Fee Payment — Actor: Student

REQ-3.1: The system shall display the due fee amount when a student selects “Pay Fee.”

REQ-3.2: The system shall accept the student's payment details and process the transaction.

REQ-3.3: The system shall update the fee status in the database and generate a payment receipt once a payment succeeds.

REQ-3.4: The system shall display an error message and allow the student to retry when a payment fails.

REQ-3.5: The system shall send a confirmation email to the student once a payment is successfully processed.

REQ-3.6: The system shall treat receipt verification as part of the fee-payment use case, consistent with the «include» relationship in Figure 6.1.

3.4 Mess Management — Actors: Student, Admin

REQ-4.1: The system shall allow the admin to update the mess menu.

REQ-4.2: The system shall track each student's meal consumption on a daily basis.

REQ-4.3: The system shall calculate the student's total mess bill and generate a mess report at the end of each billing month.

REQ-4.4: The system shall notify a student once their monthly mess bill has been generated.

REQ-4.5: The system shall allow a student to view their current mess bill at any time.

3.5 Administration and Reporting — Actor: Admin

REQ-5.1: The system shall allow the admin to manage user accounts (view, update or deactivate student and admin records).

REQ-5.2: The system shall allow the admin to generate reports covering room occupancy, fee collection and mess consumption.

REQ-5.3: The system shall allow the admin to maintain hostel-level details, including adding rooms and updating the registered warden.

4. External Interface Requirements

4.1 User Interfaces

A mobile application (Android/iOS) is the primary interface for students, covering registration, login, room application, fee payment and mess-bill viewing. A browser-based interface is used by the admin from a Windows/macOS workstation for approvals, room assignment, mess-menu updates and reporting. Both interfaces reach the same Interface/API layer, so student and admin actions ultimately arrive at the same Room, Fee and Mess controllers (Figure 6.4).

4.2 Hardware Interfaces

No special hardware is required on the client side beyond a smartphone or PC capable of running a modern browser or the HMS app. Server-side hardware must be sufficient to run Apache Tomcat (application server) and MySQL 8.0 (database server) as two separate nodes, as shown in the deployment diagram (Figure 6.6).

4.3 Software Interfaces

- Apache Tomcat — execution environment for the Auth, Room and Fee controllers.
- MySQL 8.0 (Hostel_DB) — accessed over JDBC through a dedicated DB Connection Manager; no controller queries the database directly.
- SMTP service — required to send payment-confirmation emails (REQ-3.5).

4.4 Communication Interfaces

- Client-to-server traffic uses HTTPS over port 443, passing through a firewall/load balancer.
- Application-to-database traffic uses JDBC over a secured internal connection.

5. Non-Functional Requirements

5.1 Performance

The system shall return room-availability and mess-bill queries within a few seconds under normal load, and shall support concurrent access from multiple students during peak application periods (start of a semester) without blocking the admin's workflow.

5.2 Security

Passwords shall not be stored in plain text. All client-server communication shall be encrypted (HTTPS). The system shall enforce the Student/Admin role distinction already captured in the class diagram, so a student cannot reach admin-only operations such as Manage Users or Generate Reports.

5.3 Reliability and Availability

Fee-payment updates (REQ-3.3) and room-status updates (REQ-2.5) shall be written to the database as atomic operations, so that a failure part-way through a transaction cannot leave a room marked Occupied without a corresponding successful payment, or the reverse.

5.4 Usability

Given that the Admin user class is not assumed to be technical, the admin dashboard shall present room availability, pending applications and reports without requiring any direct database or configuration knowledge.

5.5 Maintainability and Scalability

Because application logic is split into independent controllers behind their own interfaces (Figure 6.7), a module — Mess, for instance — can be modified or redeployed without changing the Auth, Room or Fee controllers, provided the API contract each exposes is kept stable.

6. System Design — UML Models

This section presents the structural and behavioural models developed for the Hostel Management System. The models are cross-referenced: the actors and use cases in 6.1 drive the workflows in 6.2, which are realised by the classes in 6.3 and the object interactions in 6.4; 6.5 shows how a single Room instance behaves over its lifecycle; and 6.6/6.7 show how the modules introduced in 6.3 are packaged and deployed.

6.1 Use Case Diagram

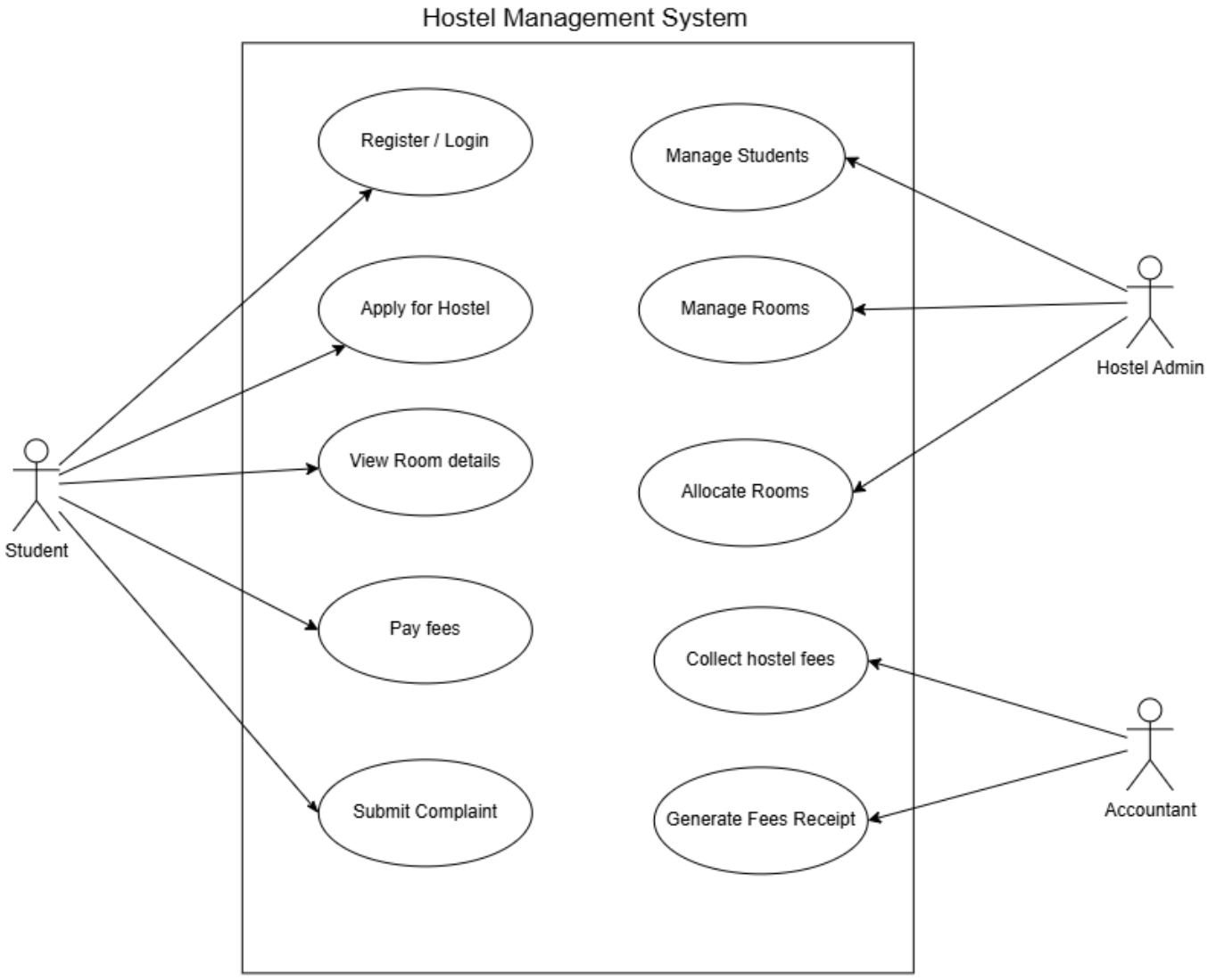


Figure 6.1 — Use Case Diagram: Hostel Management System

The system has two actors, Student and Admin, who share the Login use case. A student can register, log in, update their profile, apply for a room, make a fee payment and view the mess bill. Applying for a room is optionally extended by Select Room Preference, and making a fee payment includes Verify Payment Receipt, since a payment is not treated as complete until the receipt is confirmed. On the admin side, Login is followed by checking room availability, managing user accounts, assigning rooms, updating the mess menu and generating reports; assigning a room includes checking room availability as a mandatory step, so an admin cannot allot a room without first confirming it is free.

6.2 Activity Diagrams

Three workflows are modelled as activity diagrams: room application and allocation, the monthly mess-billing cycle, and fee payment. Together they expand the Apply for Room, View Mess Bill and Make Fee Payment use cases from Figure 6.1 into step-by-step flows.

6.2.1 Activity Diagram — Room Application and Allocation

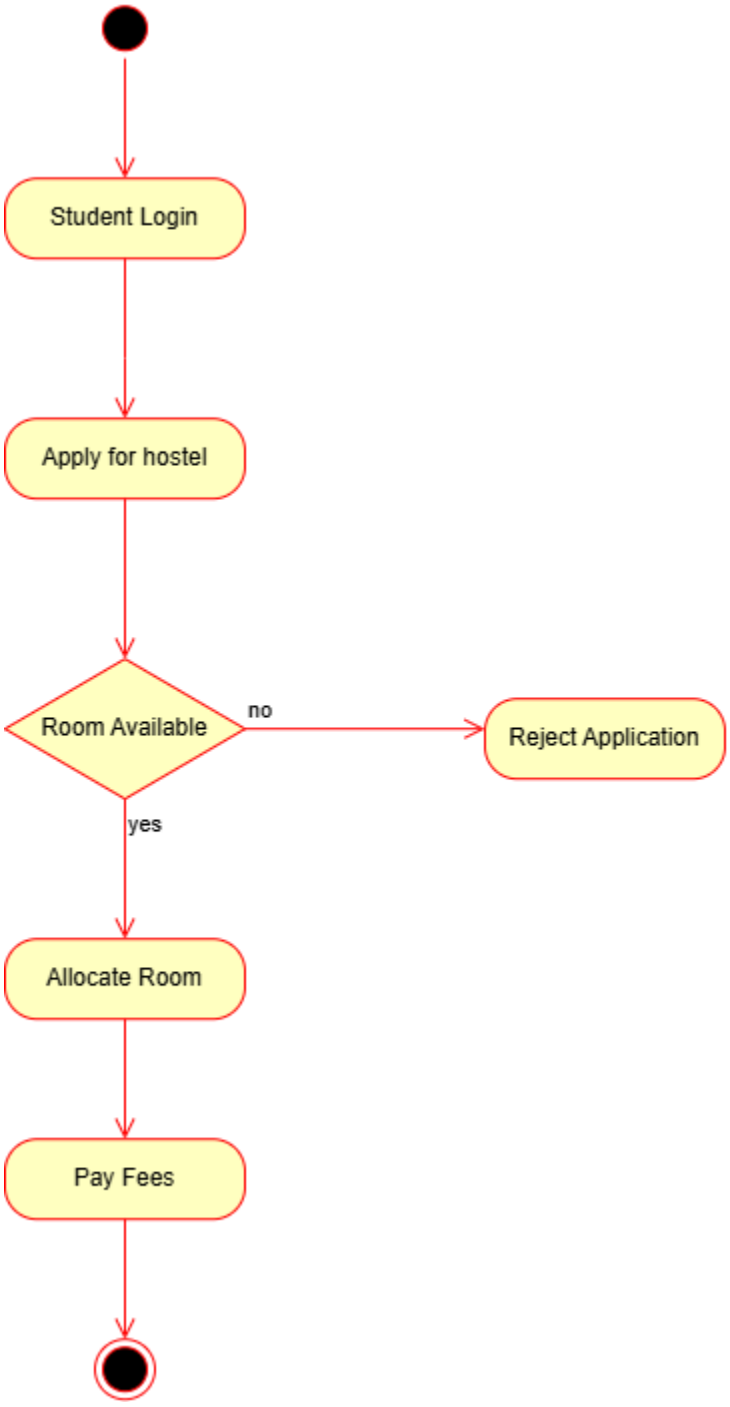


Figure 6.2.1 — Activity Diagram: Room Application and Allocation

After the student logs in and submits a room application, the admin checks room availability. If no room is available, the student is notified and the flow ends. If a room is available, the admin approves the application and the flow forks into two parallel activities: generating the fee invoice, taking the student's payment and verifying the payment receipt on one branch; updating the room's status in the database and notifying the warden on the other. The two branches join before the system confirms the final allocation, which matches REQ-2.5 through REQ-2.7.

6.2.2 Activity Diagram — Fee Payment

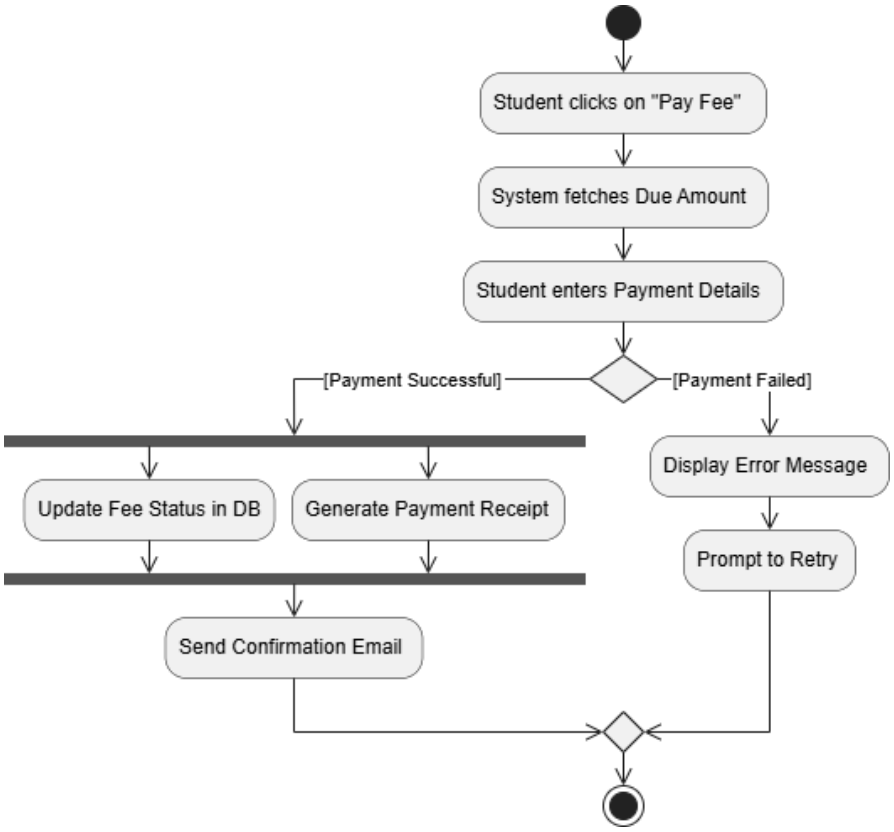


Figure 6.2.3 — Activity Diagram: Fee Payment

When a student clicks “Pay Fee,” the system fetches the due amount and the student enters payment details. On a successful payment the flow forks to update the fee status in the database and generate a payment receipt, joins, and sends a confirmation email. On a failed payment the system displays an error message and prompts the student to retry. This is the detailed realisation of REQ-3.1 through REQ-3.5.

6.3 Class Diagram

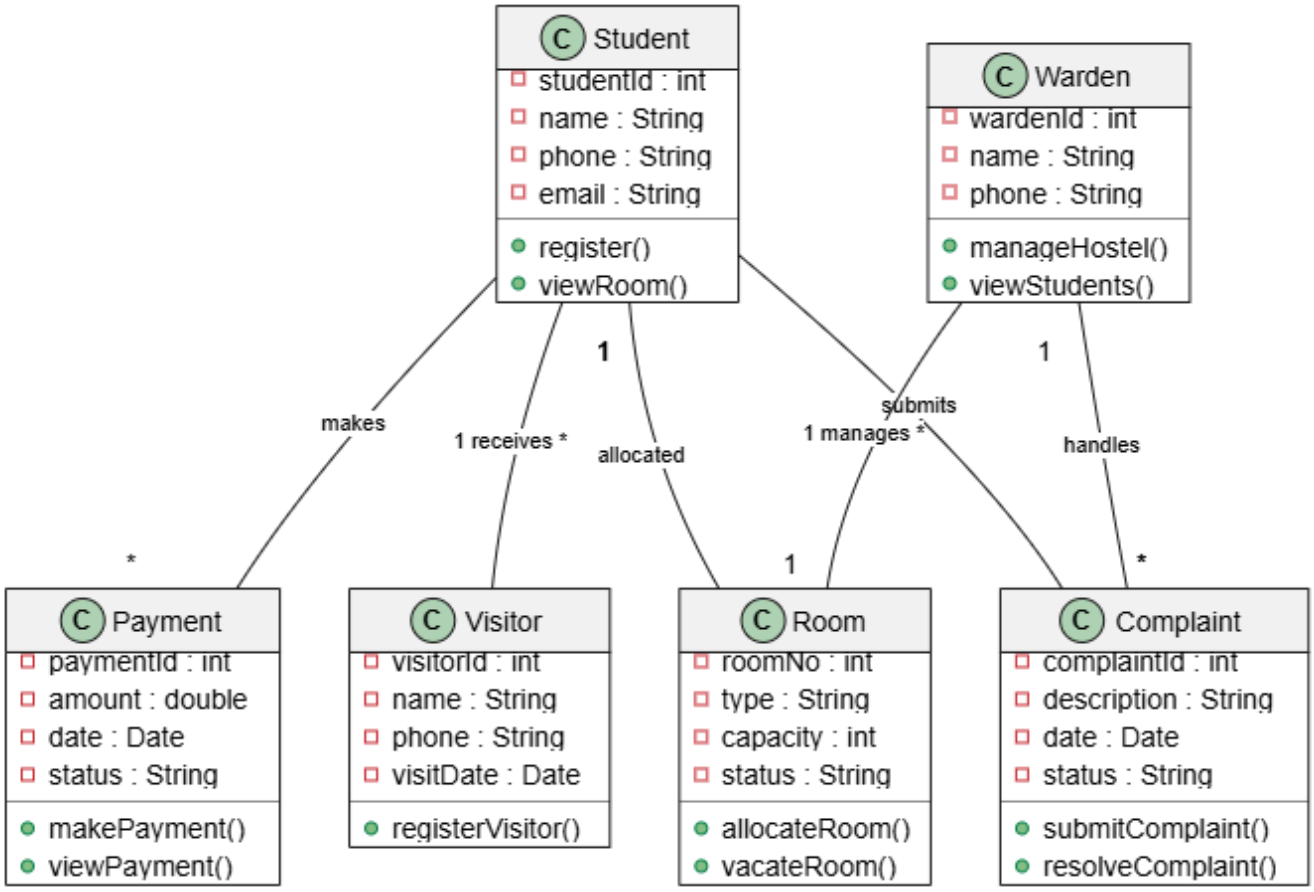


Figure 6.3 — Class Diagram: Hostel Management System

User is the base class, carrying the attributes and the login()/logout() behaviour common to every account, and is specialised into Student and Admin. Student adds studentID, course and roomID together with register(), updateProfile() and viewMessBill(); Admin adds adminID and role together with manageUser(), assignRoom() and generateReports(). A Student is linked one-to-one with a Mess account, is associated with Fee for each payment record raised against them, and is associated with Room once allocated — a Room, in turn, can hold more than one Student up to its capacity. Hostel is associated with Admin and aggregates the hostel's Room records, exposing getDetails(), addRoom() and updateWarden(). These classes map directly onto the Room, Fee, Mess and Auth controllers used throughout Section 6.

6.4 Sequence Diagram

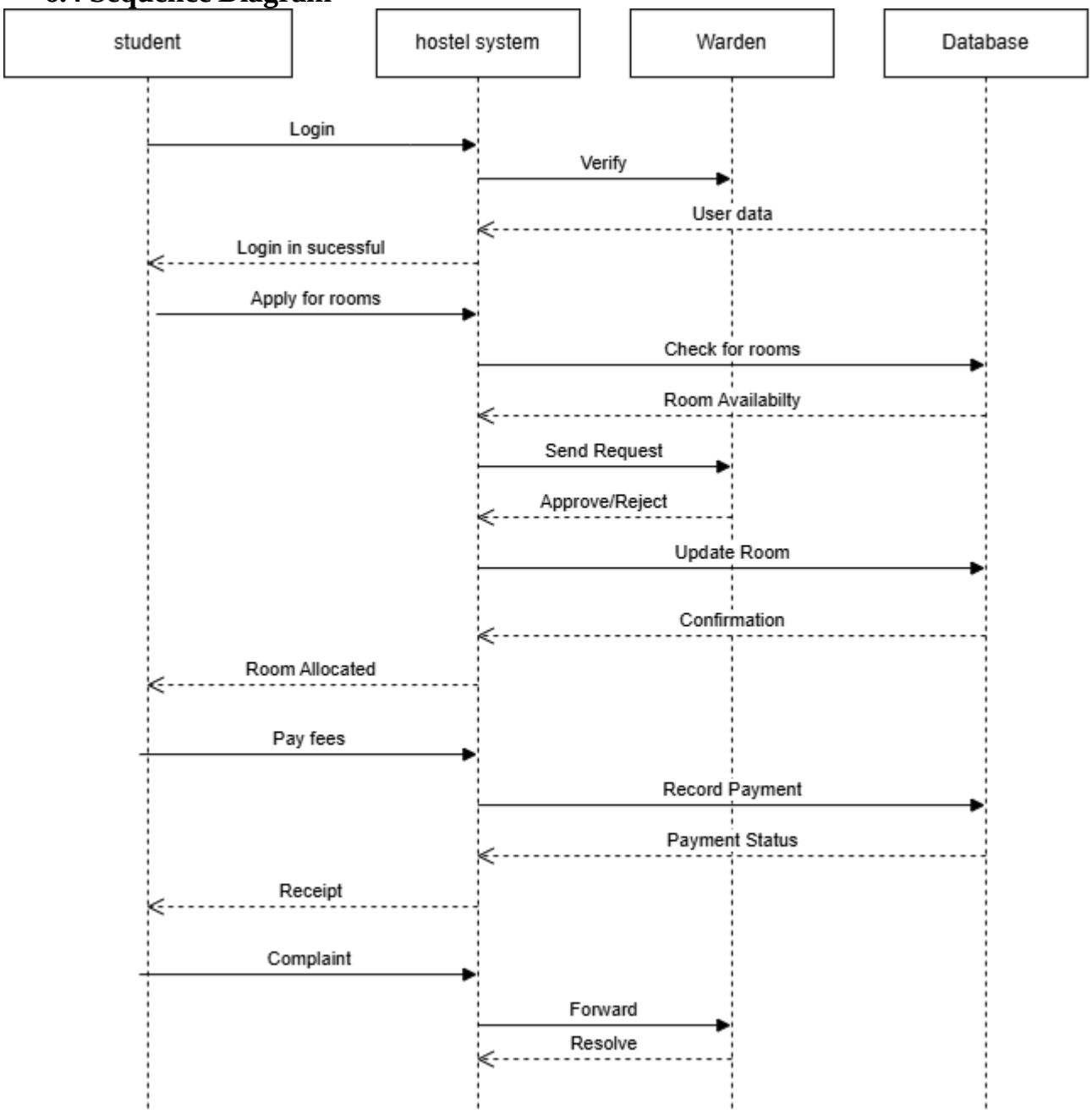


Figure 6.4 — Sequence Diagram: Room Check, Allocation and Fee Payment

Six lifelines — Student, Admin, Interface, Room Controller, Fee Controller and Database — run through three framed interactions. In Room Check, the student's request for available rooms is passed by the Interface to the Room Controller, which queries the Database and returns room details back up the chain. In Allocate Room, the student submits an application that the Room Controller saves to the Database; once the admin approves it through the Interface, the Room Controller allocates the room, updates its status in the Database, and the result is shown back to both actors. In Fee Payment, the student's payment is passed to the Fee Controller, which updates the fee record in the Database and returns a receipt that the Interface displays. This diagram is the

object-level realisation of the Apply for Room, Assign Room and Make Fee Payment use cases from Figure 6.1.

6.5 State Diagram

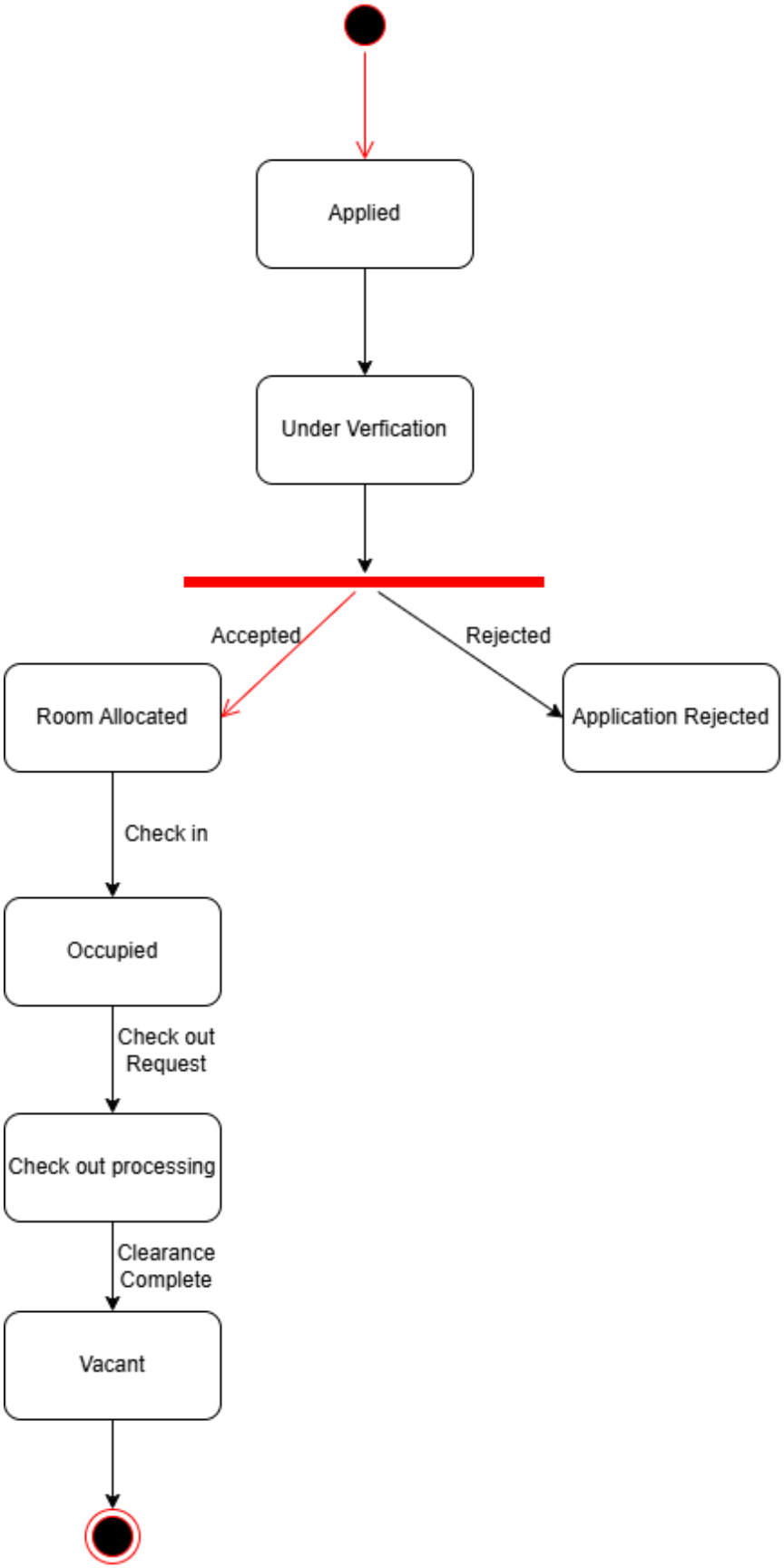


Figure 6.5 — State Diagram: Room Lifecycle

This diagram tracks a single Room instance through its lifecycle. A room starts Available (status set to “Empty”). Submitting an application moves it to Under Review, where the admin's decision branches the flow: on rejection the room returns to Available; on approval it moves to Pending Payment, which generates the fee invoice. From there, a failed or timed-out payment sends the room back to Available, while a successful payment moves it to Occupied, assigning the student to the room. Occupied eventually returns to Available once the room is vacated, and a final state marks the end of one complete allocation cycle. This state machine is what REQ-2.3 through REQ-2.7 and the Room class's status attribute (Figure 6.3) are built around.

6.6 Deployment Diagram

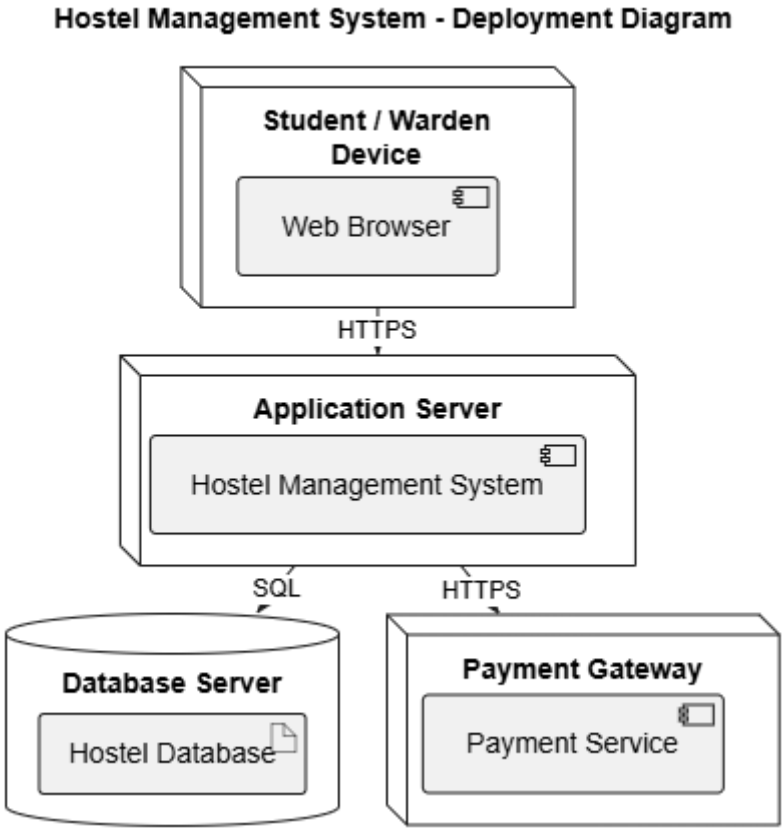


Figure 6.6 — Deployment Diagram: Hostel Management System

Students reach the system through the HMS Mobile App on an Android/iOS device, and the admin through a web browser on a Windows/macOS workstation; both connect over HTTPS to a Firewall/Load Balancer, which forwards traffic over a secure TCP connection (port 443) to the Application Server. That server runs Ubuntu Linux with Apache Tomcat as the execution environment hosting the Auth, Room and Fee controllers, each of which reaches the Database Server — a separate CentOS Linux node running MySQL 8.0 and hosting Hostel_DB — over JDBC. Splitting the application and database onto separate nodes behind a firewall is what Section 5.2 (Security) and Section 5.3 (Reliability) rely on.

6.7 Component Diagram

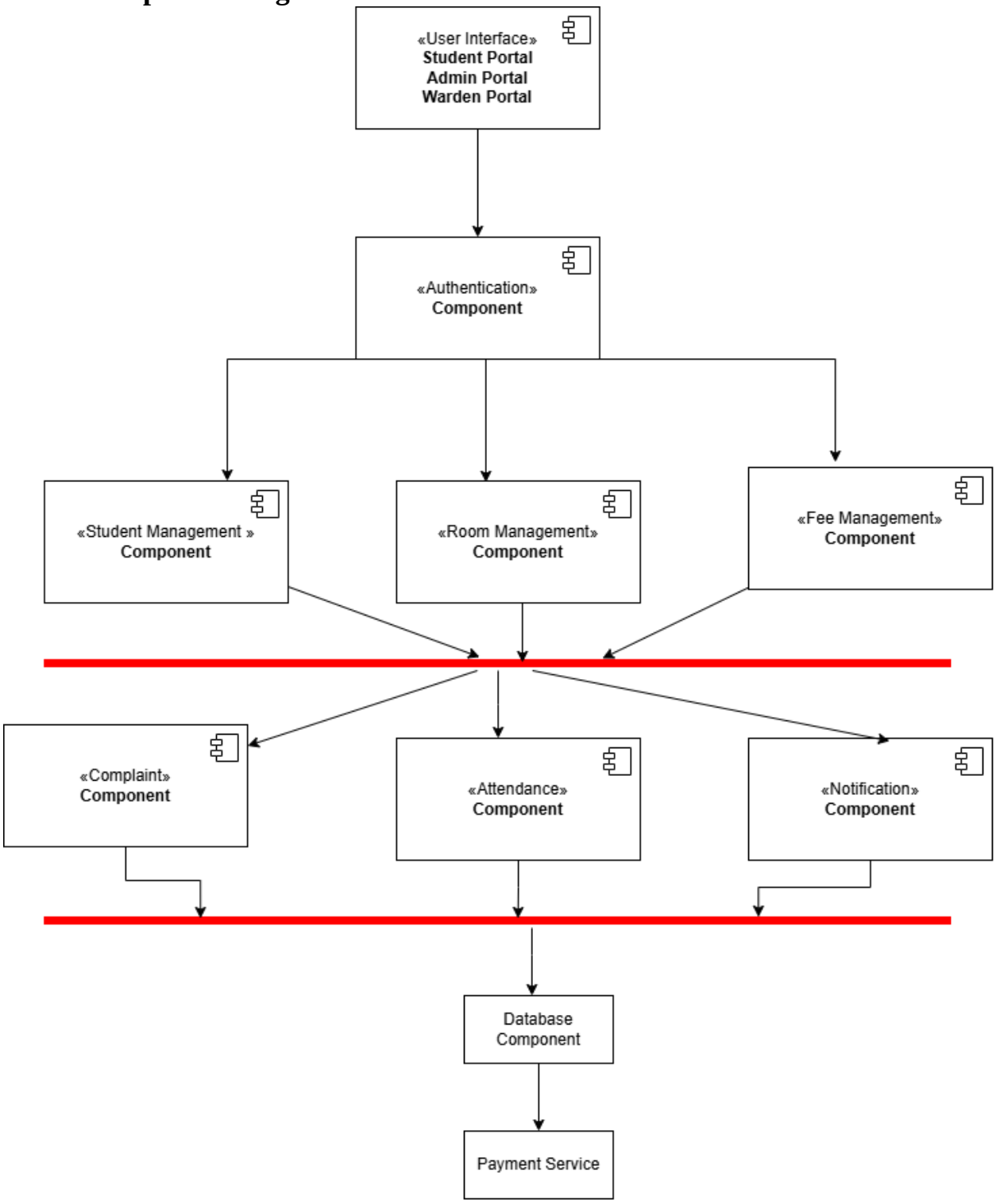


Figure 6.7 — Component Diagram: Hostel Management System

The Frontend UI (Web & Mobile) in the Client Layer uses four provided interfaces — Auth API, Room API, Fee API and Mess API — exposed by the Authentication Module, Room Controller, Fee Controller and Mess Controller in the Application Logic Layer. None of these components

talk to the database directly: each depends on a shared Database API, realised by the DB Connection Manager in the Data Access Layer, which is the only component that issues SQL/JDBC calls against Hostel_DB. This is the component-level structure that the deployment diagram (Figure 6.6) packages onto physical nodes, and that Section 2.5's design constraints are stated against.

7. Cost Estimation

7.1 Estimation Technique

Effort and schedule are estimated using the Basic COCOMO (Constructive Cost Model), since no historical productivity data exists yet for this specific team. COCOMO defines three project modes with different coefficients for the effort equation E = a × (KLOC)^b and the schedule equation D = c × (E)^d:

Project Mode	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semi-detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

HMS is classified as Organic: the team is small, the problem domain (room, fee and mess record-keeping) is well understood, requirements were stable enough to finalise the UML models in Section 6 before development, and there are no hard real-time or safety-critical constraints.

7.2 Size Estimate

Based on the five functional modules identified in Section 3 — registration/authentication, room allocation, fee payment, mess management, and admin/reporting — and the four-controller architecture in Figure 6.7, the estimated size of the system is 12,000 lines of code (12 KLOC). This is an assumption made in the absence of a completed codebase, used only to drive the estimate below; it should be replaced with an actual count once development is further along.

7.3 Effort, Schedule and Team Size

Parameter	Formula (Organic mode)	Result
Effort (E)	E = 2.4 × (KLOC)^1.05 = 2.4 × (12)^1.05	≈ 32.6 person-months
Development time (D)	D = 2.5 × (E)^0.38 = 2.5 × (32.6)^0.38	≈ 9.4 months
Average team size (P)	P = E / D = 32.6 / 9.4	≈ 3–4 people
Productivity	KLOC / E = 12 / 32.6	≈ 0.37 KLOC per person-month

7.4 Estimated Cost

Item	Value
Estimated effort (E)	32.6 person-months
Assumed cost per person-month	₹ 60,000 (fully loaded)
Estimated development cost	32.6 × ₹ 60,000 ≈ ₹ 19,56,000 (≈ ₹19.6 lakh)

The ₹60,000 per person-month figure is an assumed, fully-loaded cost (salary plus overhead) for a junior-to-mid-level developer and is stated here only as a working assumption; it should be substituted with the actual rate once known.

7.5 Relationship to the Project Schedule

The effort and duration above are a model-based estimate for the total development effort under steady, non-overlapping work. The actual plan in Section 8 reaches a shorter wall-clock duration by running backend controller development and frontend client development in parallel (Tasks 10–11) and by holding the documentation/modelling tasks (1–8) to one-week slots — a reasonable compression for a project run over a single academic semester rather than on a fully-staffed industry timeline.

8. Project Schedule

The project is planned as fifteen tasks running from late September 2026 to mid-January 2027, shown as a Gantt chart below. The first eight tasks cover scoping and the UML models in Section 6; the remaining tasks cover database setup, development, integration, testing and deployment.



Figure 8.1 — Project Gantt Chart

8.1 Key Milestones

- Design and modelling complete (Tasks 1–8): late September – mid-November 2026 — covers the SRS and all seven UML diagrams in Section 6.
- Database and core development (Tasks 9–11): mid-November – mid-December 2026 — Hostel_DB schema, the four controllers, and the web/mobile frontend, with backend and frontend work running in parallel.
- Integration and testing (Tasks 12–13): mid-December – late December 2026 — API integration followed by unit and integration testing.
- UAT and handover (Tasks 14–15): late December 2026 – mid-January 2027 — user acceptance testing with warden and student users, then production deployment and handover of the user manual.

Appendix A: Glossary of Terms

- HMS — Hostel Management System
- SRS — Software Requirements Specification
- UML — Unified Modeling Language
- JDBC — Java Database Connectivity
- KLOC — Thousand Lines of Code
- COCOMO — Constructive Cost Model
- UAT — User Acceptance Testing
- Warden — the hostel staff member who holds the Admin role in this system